INFORME DE AUDITORIA

Carlos Munaiz Cossio

CEU Master Ciberseguridad

Contenido

1. Tipo de auditoría. 1
1.1 Clasificación por tipo de auditoría 1
2. Metodología. ISOS 1
2.1 Marco Regulatorio 2
3. Estándar del sector y metodología de evaluación. 2
3.1 OWASP TOP 10 2025 2
3.2 Cvss 2
3.3 Cve 3
4. Resumen ejecutivo. 3
4.1 Resumen de vulnerabilidades 3
5.1.1 01: Brute Force 3
5.2 02: Command File 3
5.3 03: File Inclusión 3
5.4 04: File Upload 3
5.5 05: SQL Injection (Blind) 3
5.6 06: Criptografía 3
5. Vulnerabilidades 4
5.1 01: Brute Force 4
5.1.1 Evaluación 4
5.1.2 Descripción 4
5.1.3 Evidencias 4
5.1.4 Recomendaciones 6
5.1.5 Relacionadas 7
5.2 02: Command File 7
5.2.1 Evaluación 7
5.2.2 Descripción 7
5.2.3 Evidencias 7
5.2.4 Recomendaciones 9
5.2.5 Relacionadas 9
5.3 03: File Inclusión 10
5.3.1 Evaluación 10
5.3.2 Descripción 10
5.3.3 Evidencias 10
5.3.4 Recomendaciones 12
5.3.5 Relacionadas 13
5.4 04: File Upload 13

5.4.4 Evaluacion 13
5.4.2 Descripción 13
5.4.3 Evidencias 13
5.4.4 Recomendaciones 15
5.4.5 Relacionadas 15
5.5 05: SQL Injection (Blind) 15
5.5.1 Evaluacion 15
5.5.2 Descripción 15
5.5.3 Evidencias 15
5.5.4 Recomendaciones 16
5.5.5 Relacionadas 17
5.6 06: Criptografía 17
5.6.1 Evaluación 17
5.6.2 Descripción 17
5.6.3 Evidencias 17
5.6.4 Recomendaciones 18
5.6.5 Relacionadas 19

1. Tipo de auditoría.

La auditoría que se ha realizado es un pentesting técnico ofensivo, el objetivo de esta auditoria ha sido identificar y explotar las vulnerabilidades de DVWA (Damn Vulnerable Web Application) está configurada en el nivel de seguridad bajo (Low Security).

1.1 Clasificación por tipo de auditoría

Cada auditoria de seguridad se clasifica según el nivel de información previa disponible para el auditor. Existen las cajas: Negras, blancas y grises:

Caja Negra (Black Box):

El auditor no dispone de información previa del sistema. El atacante simula un ataco externo sin acceso a código ni credenciales.

Caja Blanca (White Box):

El auditor tiene acceso completo al sistema, es decir: Al código fuente, a la arquitectura y a credenciales. Esto permite un análisis exhaustivo y profundo.

Caja Gris (Grey Box):

El auditor dispone de información parcial. Puede conocer el sistema o tener acceso limitado, pero no acceso completo al código.

La auditoria realizada sobre DVWA se clasifica como Caja Blanca dado que:

- El entorno esta desplegado por el propio auditor en Kali Linux.
- Se tuvo acceso al directorio /var/www/html/dvwa.
- Se accedieron a carpetas internas como hackeable y vulnerabilities.
- Se ejecuto el proceso create/database.
- Había acceso a archivos php y configuraciones.

2. Metodología. ISOS

La auditoria se ha desarrollado siguiendo principios alineados con los estándares internacionales de gestión de seguridad.

A pesar de que es un entorno de laboratorio, el proceso se estructura conforme a:

ISO/IEC 27001 – Sistema de Gestión de Seguridad de la información

ISO/IEC 27002 – Controles de seguridad

ISO/IEC 27005 – Gestión del riesgo

Las fases que he aplicado han sido:

1. Identificación de vulnerabilidades.
2. Explotación controlada.
3. Evaluación del impacto.
4. Propuesta de medidas correctivas.

La metodología aplicada sigue un enfoque basado en riesgo, priorizando aquellas vulnerabilidades que presentan mayor impacto potencial sobre la confidencialidad, integridad y disponibilidad del sistema.

Aunque se trata de un entorno de laboratorio, se ha simulado un escenario realista de auditoría técnica siguiendo buenas prácticas del sector

2.1 Marco Regulatorio

La presente auditoría se ha realizado en un entorno controlado de laboratorio, concretamente sobre la aplicación Damn Vulnerable Web Application (DVWA) desplegada en localhost dentro de Kali Linux, con fines exclusivamente académicos.

Al tratarse de un entorno aislado y sin tratamiento de datos personales reales, no resulta de aplicación directa normativa como el Reglamento General de Protección de Datos (RGPD) o la Ley Orgánica 3/2018 (LOPDGDD). No obstante, dichas normativas se consideran marco teórico de referencia en auditorías reales donde exista tratamiento de información sensible.

En un escenario profesional, este tipo de auditoría técnica estaría regulado por:

- **Reglamento (UE) 2016/679 (RGPD).**
- **Ley Orgánica 3/2018 de Protección de Datos y Garantía de los Derechos Digitales.**
- **Esquema Nacional de Seguridad (ENS).**
- **Directiva NIS2 sobre medidas para un alto nivel común de ciberseguridad en la Unión Europea.**

La actividad desarrollada en esta práctica se ha realizado bajo principios de hacking ético, con autorización expresa y en un entorno no productivo.

3. Estándar del sector y metodología de evaluación.

Para la clasificación y evaluación de las vulnerabilidades detectadas, se han utilizado estándares ampliamente reconocidos en el ámbito de la ciberseguridad. Esto permite estructurar el análisis conforme a criterios objetivos y alineados con las buenas prácticas del sector.

La combinación de OWASP, CVSS y CVE garantiza un análisis alineado con estándares internacionales y facilita la priorización de riesgos.

3.1 OWASP TOP 10 2025

La clasificación de las vulnerabilidades identificadas se ha realizado tomando como referencia el marco OWASP Top 10 2025, que recoge las principales categorías de riesgos en aplicaciones web.

OWASP proporciona una categorización estructurada de vulnerabilidades frecuentes como inyección, control de acceso roto, fallos criptográficos o fallos de autenticación, permitiendo contextualizar cada hallazgo dentro de un riesgo reconocido internacionalmente.

En el presente análisis, cada vulnerabilidad detectada en DVWA ha sido mapeada a su categoría correspondiente dentro del OWASP Top 10.

3.2 Cvss

Para la evaluación de la severidad de las vulnerabilidades se ha utilizado el sistema Common Vulnerability Scoring System (CVSS), que permite asignar una puntuación objetiva basada en métricas como:

- Vector de ataque.
- Complejidad del ataque.
- Privilegios requeridos.
- Impacto sobre confidencialidad, integridad y disponibilidad.

La puntuación obtenida permite clasificar cada vulnerabilidad como baja, media, alta o crítica.

3.3 Cve

En aquellos casos en los que la vulnerabilidad identificada esté documentada oficialmente, se ha utilizado la referencia Common Vulnerabilities and Exposures (CVE), que proporciona un identificador único para vulnerabilidades conocidas.

El uso de referencias CVE permite contextualizar los hallazgos dentro del ecosistema global de seguridad y facilita su trazabilidad técnica.

4. Resumen ejecutivo.

4.1 Resumen de vulnerabilidades

5.1.1 01: Brute Force

Vulnerabilidad	Ausencia de limitación de intentos en autenticación.
Riesgos Asociados	Acceso no autorizado y compromiso de credenciales.
Mitigaciones	Limitar intentos, bloqueo temporal y MFA.

5.2 02: Command File

Vulnerabilidad	Ejecución de comandos del sistema por falta de validación.
Riesgos Asociados	Compromiso total del servidor.
Mitigaciones	Validación estricta y uso de listas blancas.

5.3 03: File Inclusión

Vulnerabilidad	Inclusión de archivos mediante manipulación de parámetros.
Riesgos Asociados	Acceso a archivos sensibles.
Mitigaciones	Restringir rutas y validar entradas.

5.4 04: File Upload

Vulnerabilidad	Subida de archivos sin control adecuado.
Riesgos Asociados	Ejecución remota de código.
Mitigaciones	Validar tipo de archivo y almacenar fuera del directorio público.

5.5 05: SQL Injection (Blind)

Vulnerabilidad	Manipulación de consultas SQL mediante entrada no validada.
Riesgos Asociados	Extracción y alteración de datos.
Mitigaciones	Consultas parametrizadas y control de privilegios.

5.6 06: Criptografía

Vulnerabilidad	Uso de mecanismos criptográficos débiles o almacenamiento inseguro de credenciales.
Riesgos Asociados	Exposición de contraseñas y compromiso de cuentas.

Mitigaciones	Uso de hash seguro (bcrypt/Argon2), salting y cifrado robusto de datos sensibles.

5. Vulnerabilidades

5.1 01: Brute Force

5.1.1 Evaluación

Categoria	A07:2025 Authentication Failures - A10:2025 Mishandling of Exceptional Conditions)
Impacto	Crítico: Disponibilidad, Confidencialidad e Integridad
CVSS	**CVSS 3.1 Vector - 9.8 - AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H**

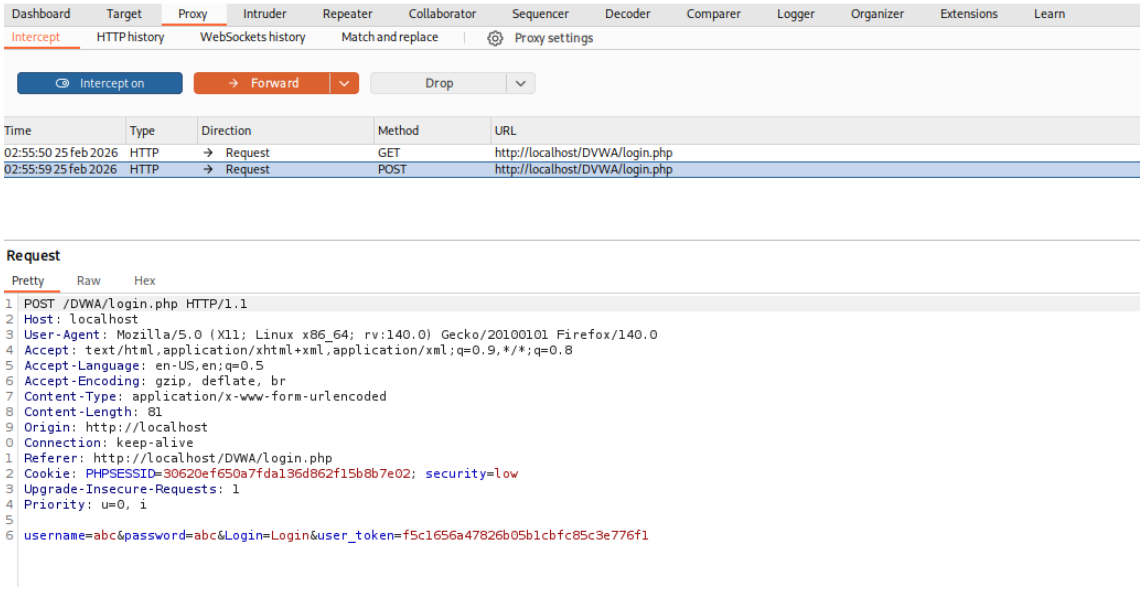
5.1.2 Descripción

La vulnerabilidad Brute Force permite realizar múltiples intentos de autenticación sin limitación.
En DVWA (nivel Low) no existe control de intentos ni mecanismos de protección, lo que permite probar combinaciones de usuario y contraseña hasta encontrar credenciales válidas.

5.1.3 Evidencias

Para la realización del ataque de fuerza bruta se utilizó Burp Suite junto con FoxyProxy correctamente configurado para interceptar el tráfico HTTP generado por el navegador.

En primer lugar, se interceptó la petición HTTP Request del formulario de autenticación de DVWA mediante la funcionalidad Proxy → Intercept de Burp Suite. Una vez capturada la petición correspondiente al intento de login, esta fue enviada al módulo Intruder para su análisis y automatización del ataque.

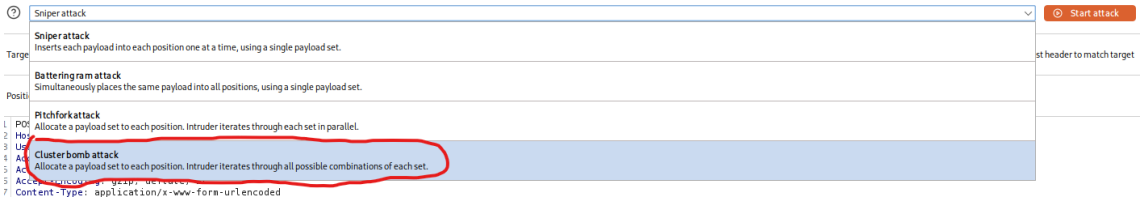


Burpsuite 5.1.3 fuerza bruta

Dentro de Intruder, se eliminaron los marcadores existentes mediante la opción Clear y posteriormente se añadieron manualmente los campos correspondientes a “username” y “password” como posiciones de ataque.

Burpsuite 5.1.3 Intruder

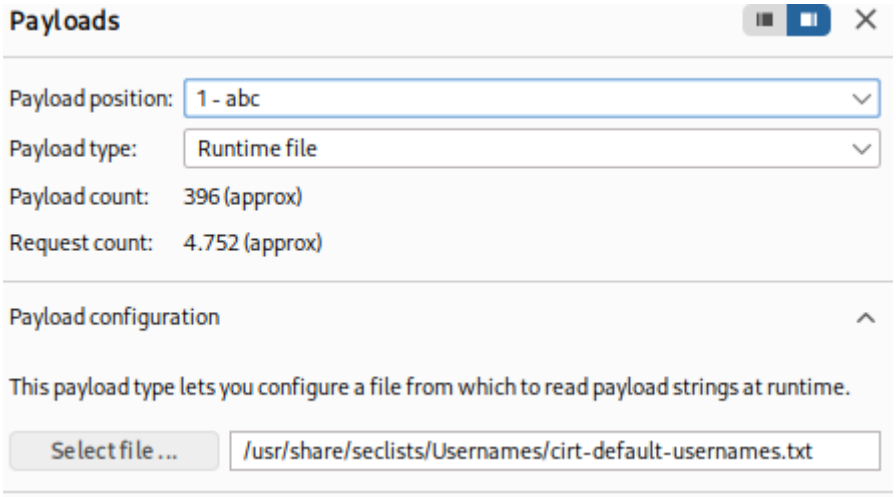
Se seleccionó el tipo de ataque **Cluster Bomb**, lo que permite probar todas las combinaciones posibles entre dos listas de datos.



Burpsuite 5.1.3 Clusterbomb attack

En este caso:

- **Payload 1**: lista de usuarios (username) procedente de una wordlist de **SecLists**.



Burpsuite Payload 1

- **Payload 2**: lista de contraseñas (password) obtenida igualmente de **SecLists**.

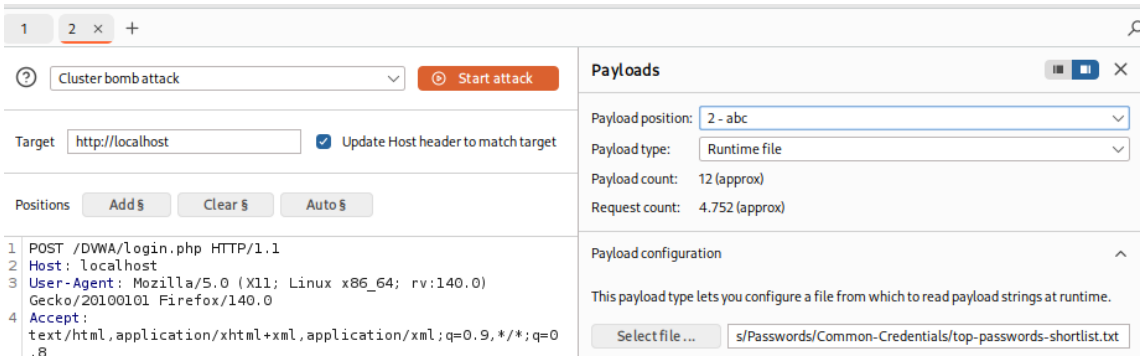
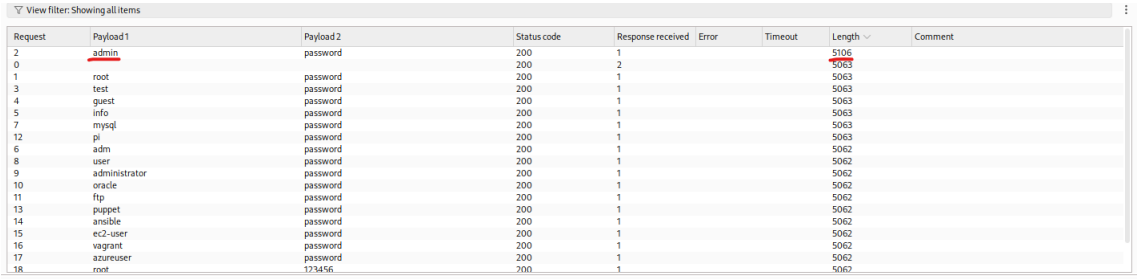


Ilustración 1 Burpsuite 5.1.3 Payload 2

Una vez configurados los payloads, se inició el ataque automatizado desde **Intruder**, permitiendo a Burp Suite generar múltiples combinaciones de credenciales.

La combinación identificada fue:

usuario: admin
contraseña: password

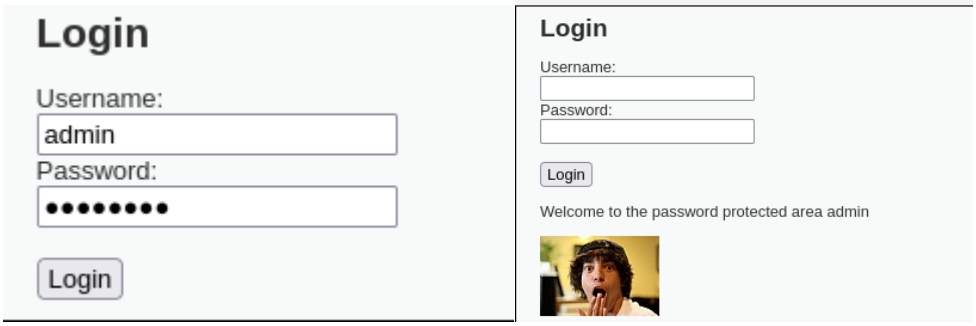


View filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
2	admin	password	200	1			5106	
0			200	2			5063	
1	root	password	200	1			5063	
3	test	password	200	1			5063	
4	guest	password	200	1			5063	
5	info	password	200	1			5063	
7	mysql	password	200	1			5063	
12	pi	password	200	1			5063	
6	adm	password	200	1			5062	
8	user	password	200	1			5062	
9	administrator	password	200	1			5062	
10	oracle	password	200	1			5062	
11	ftp	password	200	1			5062	
13	puppet	password	200	1			5062	
14	ansible	password	200	1			5062	
15	ec2-user	password	200	1			5062	
16	vagrant	password	200	1			5062	
17	azureuser	password	200	1			5062	
18	root	123456	200	1			5062	

Ilustración 2 Burpsuite 5.1.3 Ataque fuerza bruta

Finalmente, se verificó manualmente el acceso a la aplicación utilizando dichas credenciales, confirmando que el ataque de fuerza bruta había sido exitoso.



Login 5.1.3 Éxito del ataque

5.1.4 Recomendaciones

Para mitigar esta vulnerabilidad se recomienda implementar los siguientes controles de seguridad:

- Limitar el número de intentos de autenticación permitidos.
- Implementar bloqueo temporal de cuentas tras múltiples intentos fallidos.
- Incorporar mecanismos CAPTCHA para evitar ataques automatizados.

- Registrar intentos de autenticación sospechosos en los logs del sistema.
- Implementar autenticación multifactor (MFA) en cuentas críticas.
- Establecer políticas de contraseñas robustas.

Estas medidas reducen significativamente la probabilidad de éxito de ataques de fuerza bruta.

5.1.5 Relacionadas

OWASP Top 10 2025 A07: Identification and Authentication Failures

CWE CWE-307 – Improper Restriction of Excessive Authentication Attempts

CAPEC CAPEC-49 – Password Brute Forcing

5.2 02: Command File

5.2.1 Evaluación

Categoria	A05:2025 Injection - A01:2025 Broken Access Control
Impacto	Crítica: Disponibilidad, Confidencialidad e Integridad
CVSS	**CVSS 3.1 Vector – 10 - AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H**

5.2.2 Descripción

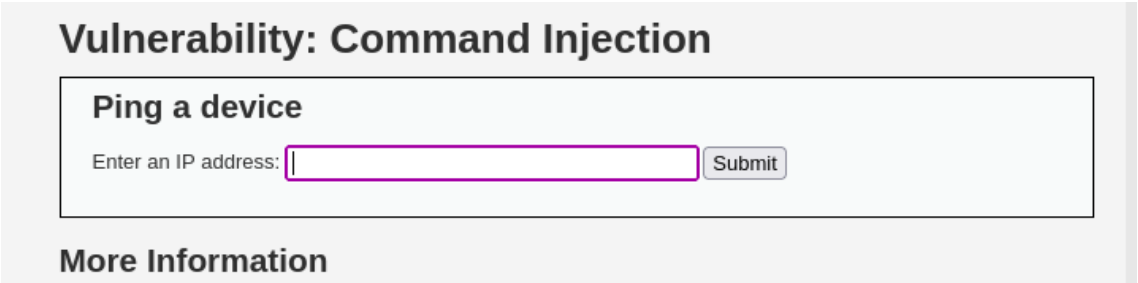
La vulnerabilidad Command Injection permite a un atacante ejecutar comandos del sistema operativo a través de una aplicación web cuando esta no valida correctamente los datos introducidos por el usuario.

En DVWA (nivel Low) el parámetro introducido en el formulario se concatena directamente a un comando del sistema sin filtrado ni validación, permitiendo ejecutar comandos adicionales en el servidor.

Esto puede comprometer la integridad y confidencialidad del sistema, ya que un atacante podría obtener información del sistema o ejecutar comandos arbitrarios.

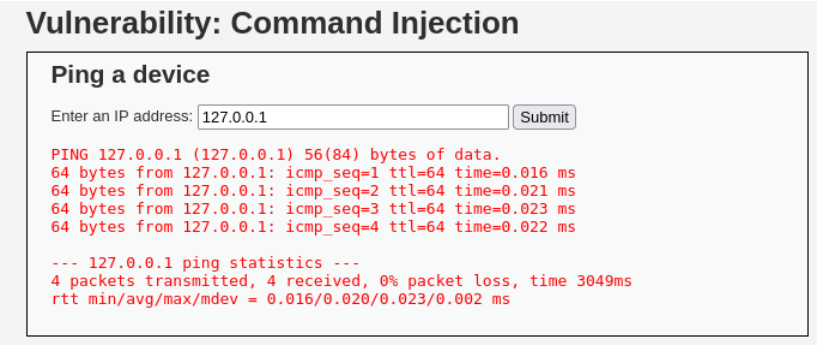
5.2.3 Evidencias

En el módulo Command Injection de DVWA (nivel Low), el valor introducido en el campo de entrada se concatena directamente a un comando del sistema sin aplicar ningún tipo de filtrado o validación. Esto permite que un atacante añada comandos adicionales que serán ejecutados por el servidor.



Formulario Ping 5.2.2

La funcionalidad original de la aplicación consiste en realizar un **ping** a la dirección IP introducida por el usuario. Sin embargo, al no existir mecanismos de validación, es posible introducir operadores de concatenación de comandos para ejecutar instrucciones arbitrarias del sistema operativo.



Ping 5.2.2 Devuelta de paquetes

Durante las pruebas se ejecutaron distintos comandos del sistema, entre ellos:

- ls -l → muestra el contenido de un directorio y sus permisos
- pwd → muestra el directorio de trabajo actual
- whoami → indica el usuario con el que se está ejecutando el proceso

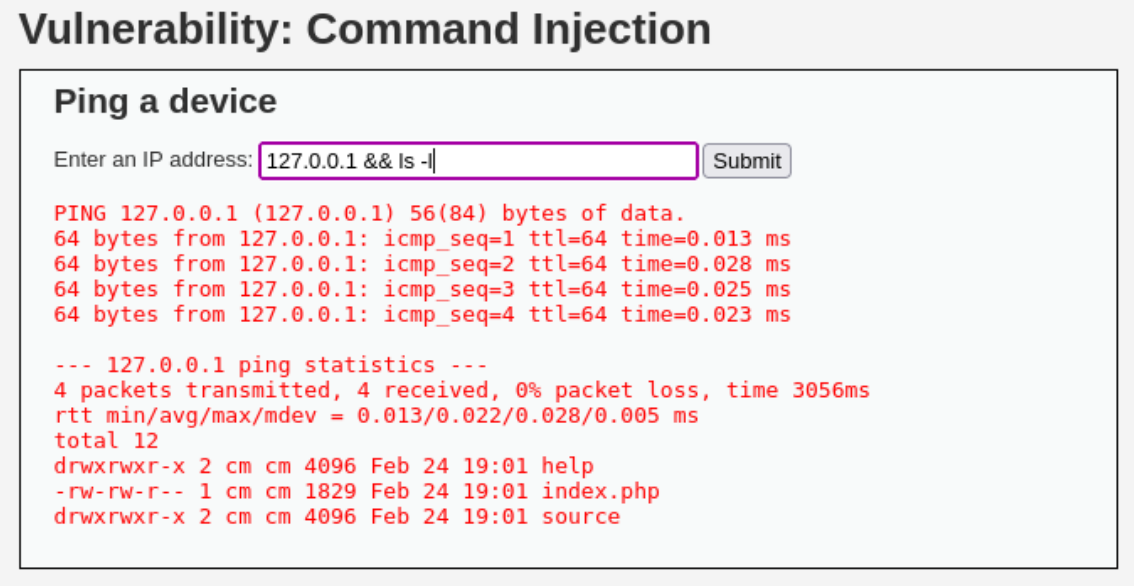
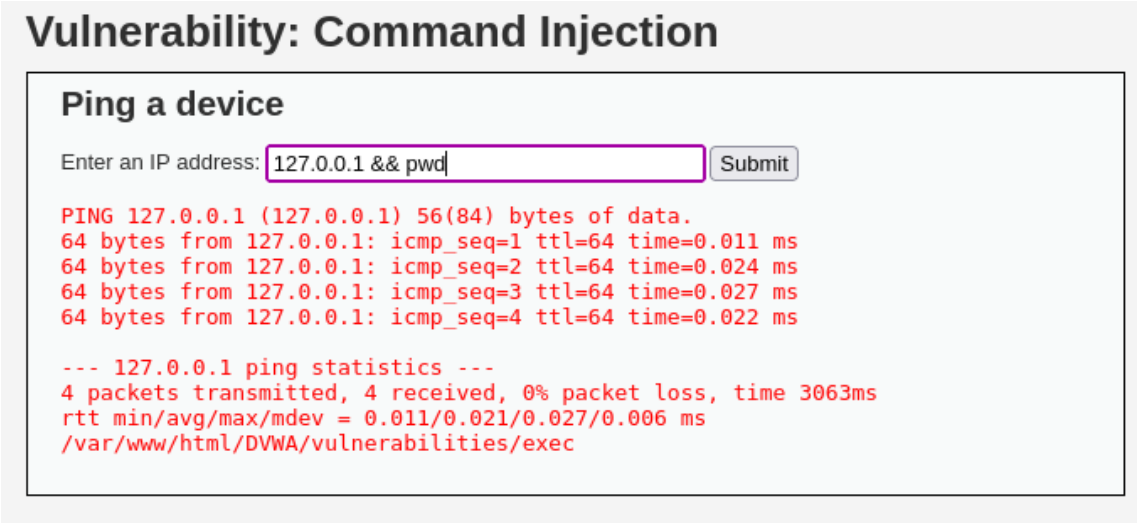


Ilustración 3 Ping 5.2.2 Comando ls -l



Formulario Ping 5.2.2 Comando pwd

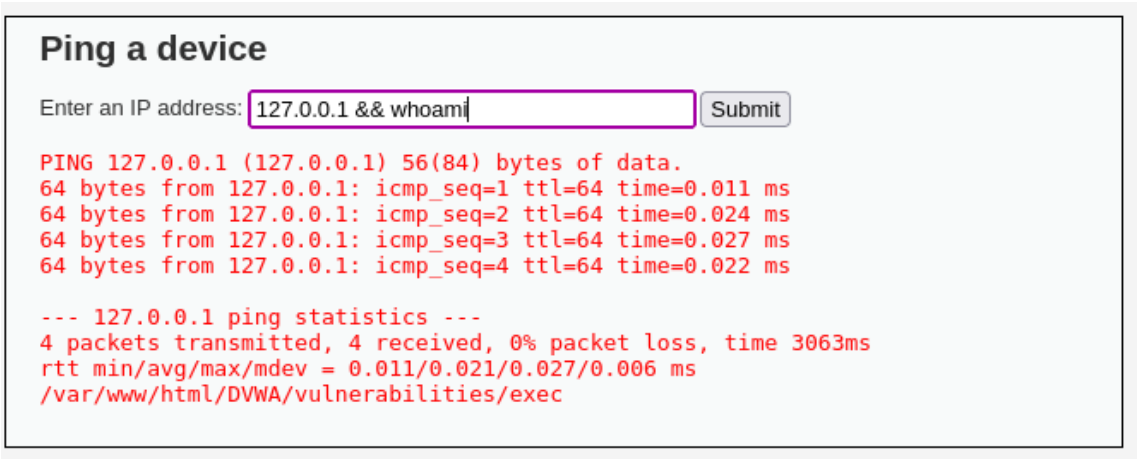


Ilustración 4 Formulario Ping 5.2.2 Comando whoami

Esto demuestra que el servidor ejecuta directamente los comandos introducidos por el usuario.

Durante la fase de pruebas también se ejecutó el comando rm -f *, lo que provocó la eliminación de archivos del sistema de la aplicación, siendo necesario restaurar posteriormente la base de datos del entorno de laboratorio.

5.2.4 Recomendaciones

Para mitigar esta vulnerabilidad se recomienda:

- Validar y sanitizar todas las entradas del usuario.
- Evitar concatenar directamente datos del usuario en comandos del sistema.
- Utilizar listas blancas (whitelisting) para los valores permitidos.
- Implementar funciones seguras del lenguaje para ejecutar comandos.
- Aplicar el principio de mínimos privilegios en el servidor.

5.2.5 Relacionadas

OWASP Top 10 A03: Injection

CWE-77 Improper Neutralization of Special Elements used in a Command

5.3 03: File Inclusión

5.3.1 Evaluación

Categoría	A01:2025 Broken Access Control - A06:2025 Insecure Design
Impacto	Alta: Disponibilidad
CVSS	**CVSS 3.1 Vector - 8.6 - AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N**

5.3.2 Descripción

La vulnerabilidad **File Inclusion** permite a un atacante incluir y ejecutar archivos en el servidor a través de parámetros de entrada que no están correctamente validados.

En el módulo **File Inclusion** de DVWA (nivel Low), el parámetro utilizado para cargar las páginas se introduce directamente en la ruta del archivo que el servidor debe mostrar. Al no existir mecanismos de validación o filtrado, es posible manipular dicho parámetro para acceder a archivos del sistema.

Mediante técnicas de **Local File Inclusion (LFI)**, un atacante puede acceder a archivos sensibles del servidor utilizando secuencias de *directory traversal*.

Por ejemplo, se pudo acceder al archivo:

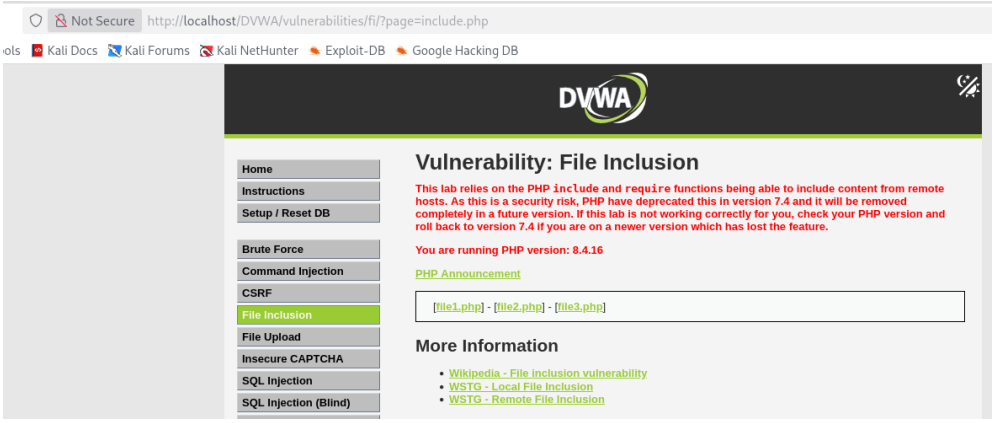
../../../../etc/passwd

Este archivo contiene información sobre los usuarios del sistema, lo que confirma que la aplicación permite acceder a archivos fuera del directorio previsto por la aplicación.

Esta vulnerabilidad puede comprometer la **confidencialidad del sistema**, permitiendo la exposición de archivos sensibles del servidor.

5.3.3 Evidencias

Durante la explotación de la vulnerabilidad se observó que la aplicación permite cargar distintos archivos a través de un parámetro en la URL. En la interfaz de la aplicación aparecen tres archivos disponibles: file1, file2 y file3.



File Inclusion 5.3.3

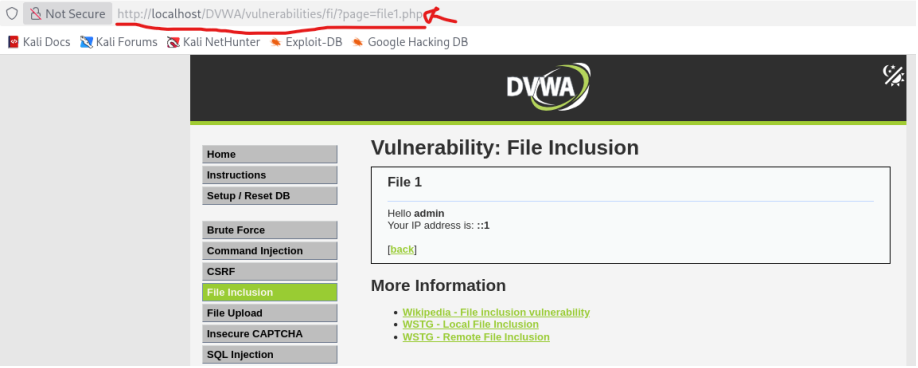
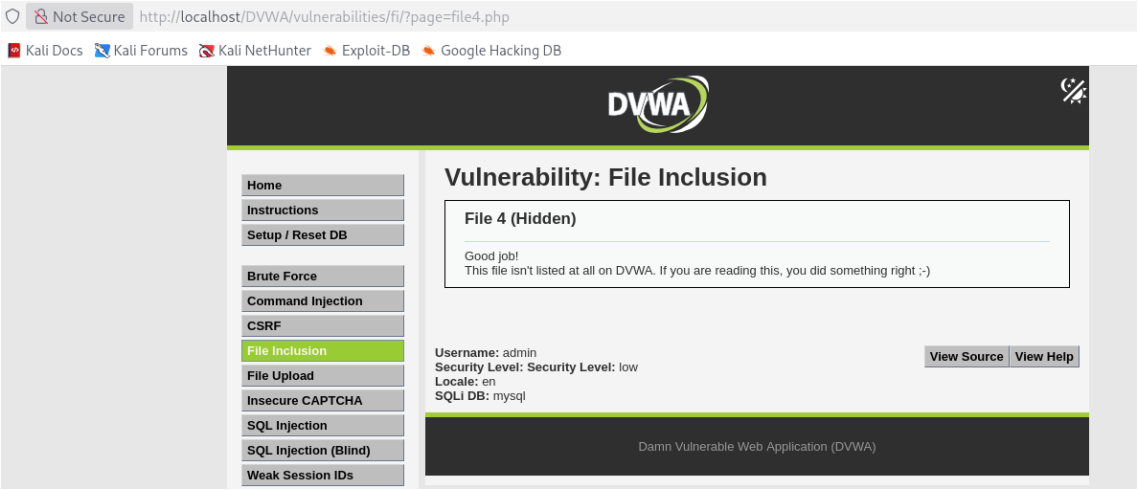


Ilustración 5 File Inclusion 5.3.3 URL

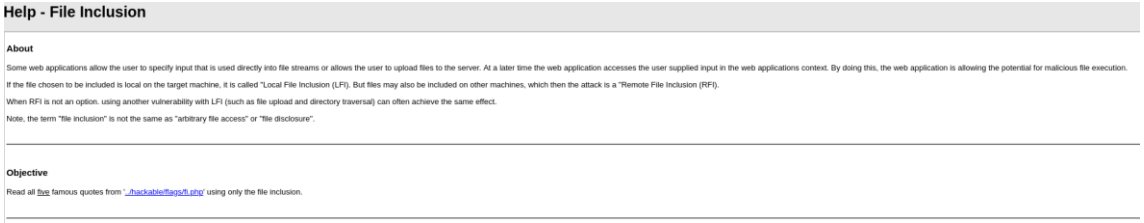
Cada uno de ellos corresponde a un valor distinto en el parámetro de la URL. Al analizar este patrón, se probó a modificar manualmente el valor del parámetro siguiendo la misma secuencia, sustituyendo file3 por file4.

Al realizar esta modificación, la aplicación mostró una página que no estaba indexada en la interfaz, indicando que era posible acceder a recursos internos simplemente manipulando el parámetro de la URL.



File Inclusion 5.3.3 File4.php

Posteriormente se consultó la sección de ayuda del módulo, donde se indicaba la posibilidad de explotar la vulnerabilidad mediante File Inclusion.



File Inclusion 5.3.3 Modulo Help

Se introdujo en la URL el payload indicado en la guía. Inicialmente la página se mostró en blanco, lo que indicaba que la ruta del archivo no estaba correctamente especificada. Esto se debía a que el archivo se encontraba en un **directorio superior** al actual.

Para solucionarlo se utilizó directory traversal (../) con el fin de navegar hacia el directorio padre y localizar el archivo correspondiente.

File Inclusion 5.3.3 url con index en blanco

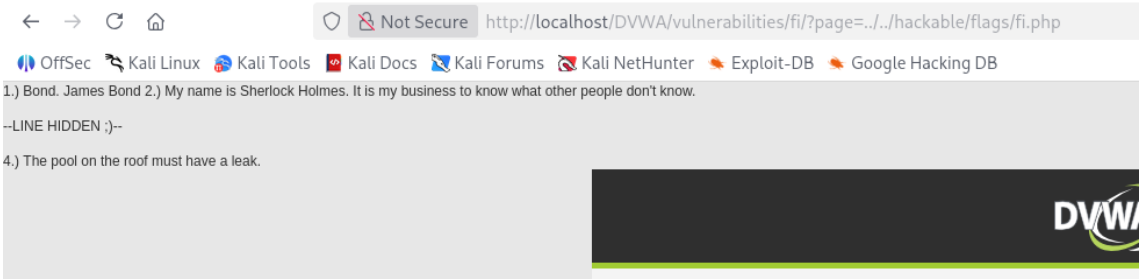


Ilustración 6 File Inclusion 5.3.3 Url corregida añadiendo../../

Adicionalmente, se comprobó que era posible acceder al archivo del sistema:

/etc/passwd

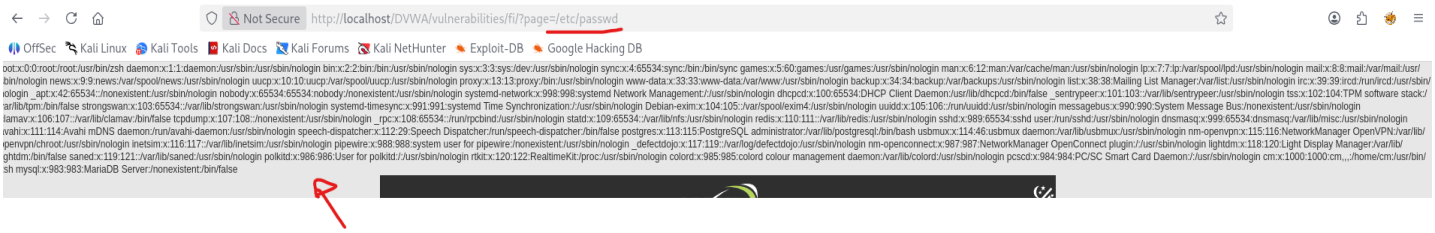
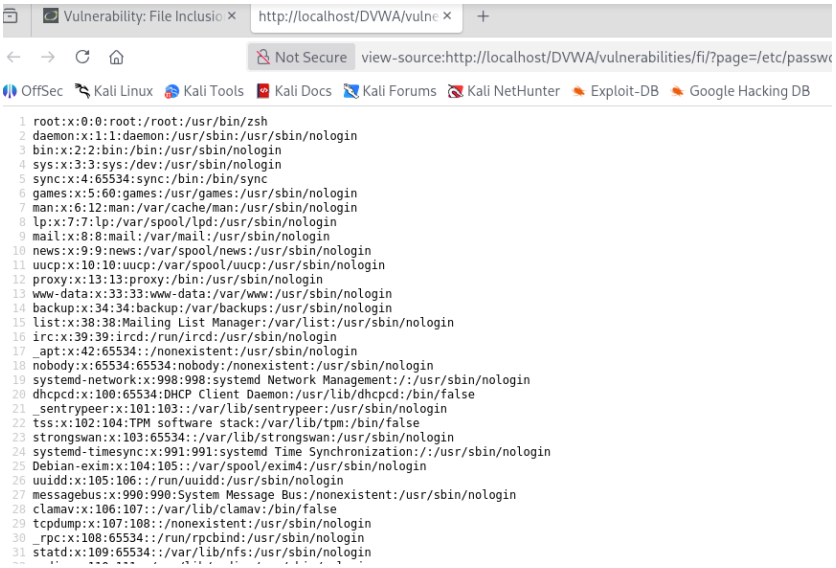


Ilustración 7File Inclusion 5.3.3 /etc/passwd

Este archivo contiene información sobre los usuarios del sistema operativo, lo que demuestra que la aplicación permite acceder a archivos sensibles del servidor.

Para visualizar el contenido de forma más clara se utilizó la opción **Ctrl + U** del navegador, permitiendo inspeccionar el código fuente de la página y facilitar la lectura de la información obtenida.



File Inclusion 5.3.3 File Inclusion 5.3.3 Ctrl + facilita la lectura

5.3.4 Recomendaciones

Para mitigar esta vulnerabilidad se recomienda implementar las siguientes medidas de seguridad:

- Validar los parámetros de entrada utilizados para cargar archivos.
- Utilizar listas blancas (whitelisting) que permitan únicamente la carga de archivos previamente definidos por la aplicación.
- Evitar el uso directo de rutas de archivos proporcionadas por el usuario.
- Implementar controles que impidan el uso de secuencias de directory traversal como ../.

- Configurar correctamente los permisos de acceso a archivos del sistema para evitar la exposición de información sensible.

La implementación de estas medidas reduce significativamente el riesgo de acceso no autorizado a archivos del servidor.

5.3.5 Relacionadas

OWASP Top 10 A05: Security Misconfiguration

CWE CWE-98 – Improper Control of Filename for Include/Require Statement

CAPEC CAPEC-126 – Path Traversal

5.4 04: File Upload

5.4.4 Evaluacion

Categoria	A01:2025 Broken Access Control - A06:2025 Insecure Design
Impacto	Crítica: Disponibilidad, Integridad e Confidencialidad
CVSS	**CVSS 3.1 Vector – 10 - AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H**

5.4.2 Descripción

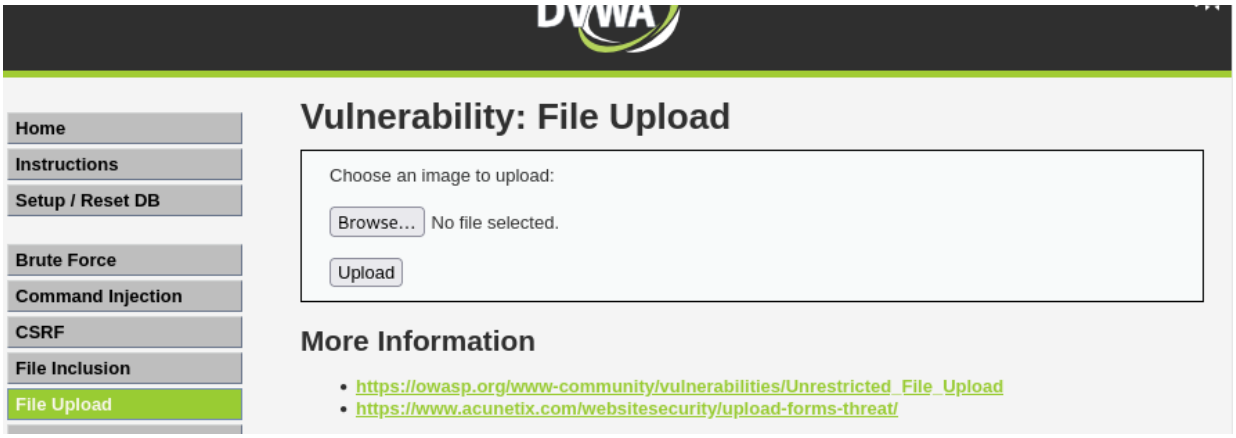
La vulnerabilidad File Upload ocurre cuando una aplicación permite subir archivos al servidor sin realizar una validación adecuada del tipo o contenido del archivo.

En el módulo File Upload de DVWA (nivel Low), la aplicación permite subir archivos al servidor sin verificar correctamente su extensión o contenido. Esto permite a un atacante subir archivos maliciosos, como scripts ejecutables, que posteriormente pueden ser ejecutados en el servidor.

Este tipo de vulnerabilidad puede permitir a un atacante ejecutar código en el servidor, comprometiendo la confidencialidad, integridad y disponibilidad del sistema.

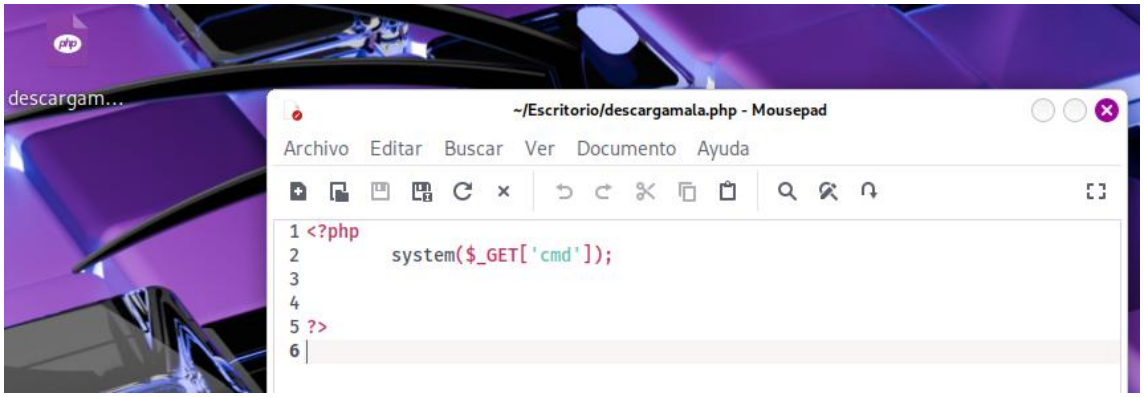
5.4.3 Evidencias

Durante la explotación de la vulnerabilidad **File Upload** se comprobó que la aplicación permite subir archivos al servidor sin realizar una validación adecuada del tipo de archivo.



File Inclusion 5.4.3 la interfaz de upload

Para demostrar la vulnerabilidad se creó un archivo malicioso denominado **descarga_mala.php**, cuyo objetivo era ejecutar comandos del sistema operativo desde el navegador.



File Inclusion 5.4.3 Archivo.php malware

Una vez creado el archivo, este fue subido a través del formulario de subida de archivos disponible en la aplicación. El servidor aceptó la subida sin aplicar ningún tipo de restricción sobre la extensión o contenido del archivo.

File Inclusion 5.4.3 Archivo.php malware descargado

Tras completarse la subida, el archivo quedó almacenado en el directorio de **uploads** de la aplicación. Accediendo directamente a la ruta del archivo desde el navegador fue posible ejecutar el código contenido en él.

Posteriormente se utilizó el parámetro:

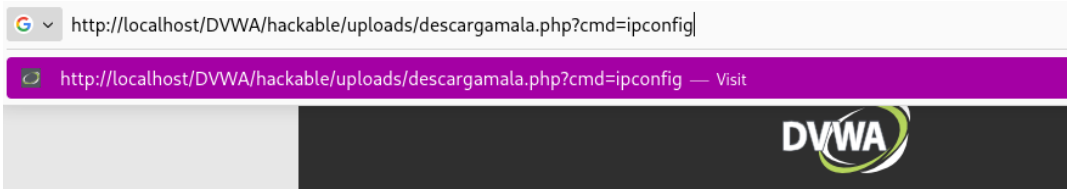
?cmd=

para enviar comandos al servidor a través de la URL.

Por ejemplo:

?cmd=ipconfig

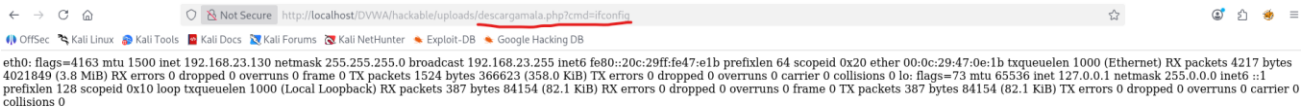
En sistemas **Windows**, este comando permite obtener información sobre la configuración de red del sistema.



File Inclusion 5.4.3 Archivo.php inyectando comandos desde cmd ejecutable

En sistemas **Linux**, el comando equivalente sería:

?cmd=ifconfig



La correcta ejecución de estos comandos confirma que el archivo subido permite ejecutar instrucciones en el servidor, demostrando la existencia de la vulnerabilidad.

5.4.4 Recomendaciones

Para mitigar esta vulnerabilidad se recomienda:

- Validar el tipo de archivo que se permite subir al servidor.
- Permitir únicamente extensiones seguras mediante listas blancas (whitelisting).
- Comprobar el contenido real del archivo, no solo su extensión.
- Almacenar los archivos subidos fuera del directorio público del servidor web.
- Renombrar los archivos subidos para evitar la ejecución directa.
- Configurar el servidor para evitar la ejecución de scripts en directorios de subida.

5.4.5 Relacionadas

OWASP Top 10 A03: Injection

CWE CWE-434 – Unrestricted Upload of File with Dangerous Type

CAPEC CAPEC-650 – Upload a Web Shell to a Web Server

5.5 05: SQL Injection (Blind)

5.5.1 Evaluacion

Categoria	A05:2025 Injection -A01:2025 Broken Access Control
Impacto	Crítica: Disponibilidad, Integridad e Confidencialidad
CVSS	**CVSS 3.1 Vector – 10 - AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H**

5.5.2 Descripción

La vulnerabilidad SQL Injection Blind ocurre cuando una aplicación web permite inyectar consultas SQL maliciosas, pero no muestra directamente los resultados de la consulta en la respuesta de la aplicación.

En este tipo de vulnerabilidad, el atacante debe inferir la información del sistema analizando el comportamiento de la aplicación, por ejemplo mediante respuestas booleanas o tiempos de respuesta del servidor.

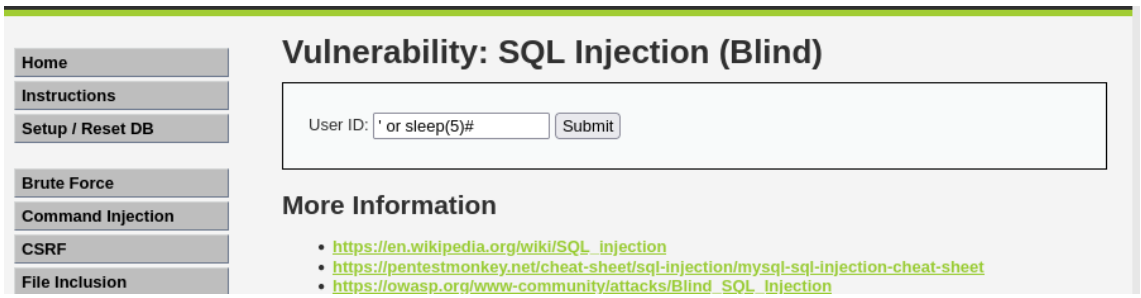
En el módulo SQL Injection Blind de DVWA (nivel Low), el parámetro introducido por el usuario se inserta directamente en una consulta SQL sin validación ni sanitización. Esto permite manipular la consulta original y obtener información de la base de datos mediante técnicas de inferencia.

5.5.3 Evidencias

En esta vulnerabilidad la aplicación no devuelve directamente los resultados de la consulta SQL, por lo que no es posible confirmar la inyección observando datos en pantalla. En su lugar, es necesario analizar el comportamiento de la aplicación, concretamente el tiempo de respuesta del servidor.

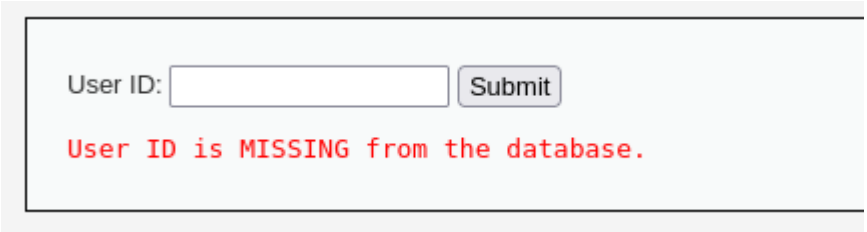
Para comprobar la existencia de la vulnerabilidad se utilizó una técnica de Time-Based Blind SQL Injection, introduciendo el siguiente payload en el parámetro vulnerable:

' OR SLEEP(5)#



File Inclusion 5.5.3 inyectando ‘ or sleep

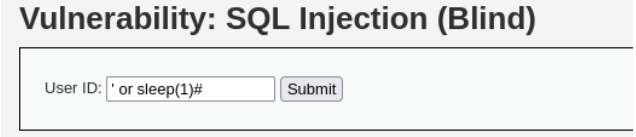
La función SLEEP() provoca una pausa en la ejecución de la consulta SQL durante el número de segundos indicado. Tras enviar la petición, se observó que la respuesta del servidor tardaba aproximadamente 5 segundos en devolverse, lo que indica que el comando SQL se estaba ejecutando correctamente en la base de datos.



File Inclusion 5.4.3 Form responde a nuestro comando

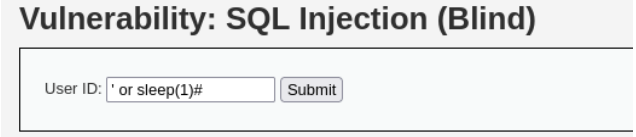
Posteriormente se realizó una segunda prueba reduciendo el tiempo de espera:

' OR SLEEP(1)#



SQL BLIND 5.5.3 otra prueba con sleep 1

En este caso la respuesta del servidor se retrasó aproximadamente 1 segundo, confirmando nuevamente que la consulta SQL estaba siendo ejecutada por la base de datos.



SQL BLIND 5.5.3 sleep 1

Este comportamiento demuestra que el parámetro es vulnerable a SQL Injection Blind basada en tiempo, ya que es posible manipular la consulta SQL y observar su ejecución mediante el tiempo de respuesta del sistema.

5.5.4 Recomendaciones

Para mitigar esta vulnerabilidad se recomienda:

- Utilizar consultas preparadas (Prepared Statements) o parametrizadas.
- Validar todas las entradas proporcionadas por el usuario.
- Implementar ORM o frameworks seguros que gestionen las consultas SQL.
- Limitar los permisos de las cuentas utilizadas para acceder a la base de datos.
- Implementar mecanismos de monitorización y registro de consultas sospechosas.

5.5.5 Relacionadas

OWASP Top 10 A03: Injection

CWE CWE-89 – Improper Neutralization of Special Elements used in an SQL Command

CAPEC CAPEC-66 – SQL Injection

5.6 06: Criptografía

5.6.1 Evaluación

Categoria	A04:2025 Cryptographic Failures - A07:2025 Authentication Failures
Impacto	Alta: Confidencialidad
CVSS	**CVSS 3.1 Vector- 7.5** - AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

5.6.2 Descripción

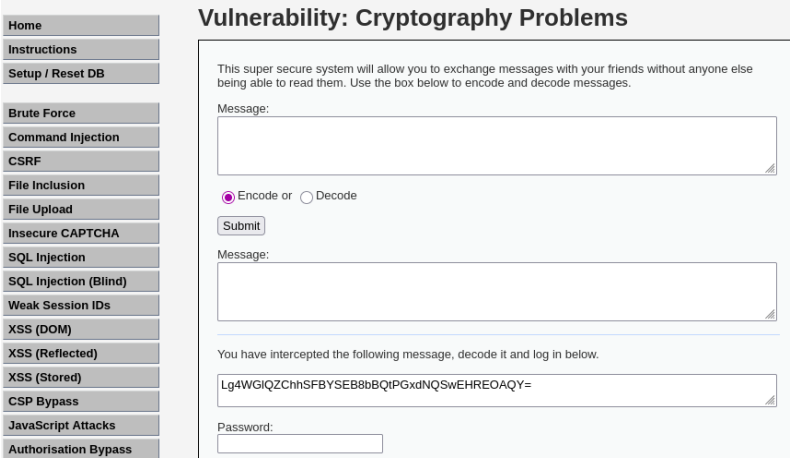
La vulnerabilidad relacionada con **criptografía** aparece cuando una aplicación utiliza algoritmos criptográficos débiles o implementaciones inseguras para proteger información sensible.

En el caso de DVWA, las contraseñas de los usuarios se almacenan mediante **hashes débiles**, lo que permite que un atacante pueda recuperar las contraseñas originales utilizando técnicas como **ataques de diccionario o tablas rainbow**.

El uso de algoritmos criptográficos obsoletos o insuficientemente robustos puede comprometer la seguridad de las credenciales almacenadas en la base de datos.

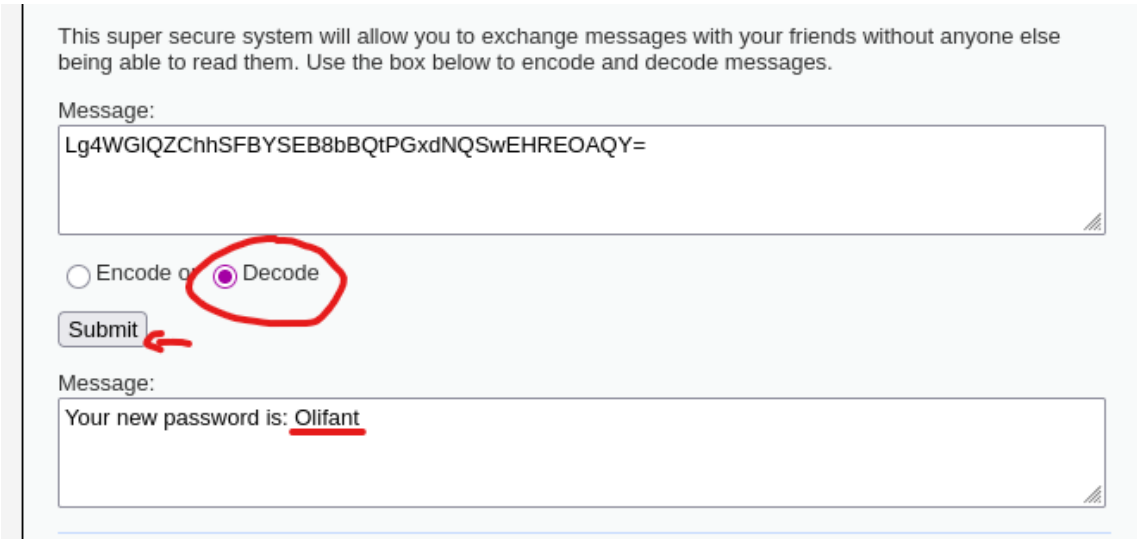
5.6.3 Evidencias

En este ejercicio se debía 4 una contraseña que se encontraba codificada en el tráfico interceptado.



Cripologia 5.6.3

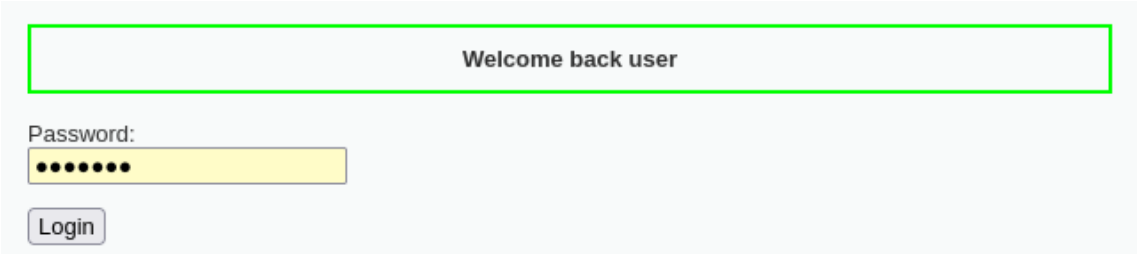
Durante el análisis se capturó el mensaje que contenía la contraseña codificada. Tras observar su formato, se identificó que el contenido seguía el patrón típico de codificación Base64, un método de codificación utilizado para representar datos binarios en formato texto.



Criptologia 5.6.3 Base 64

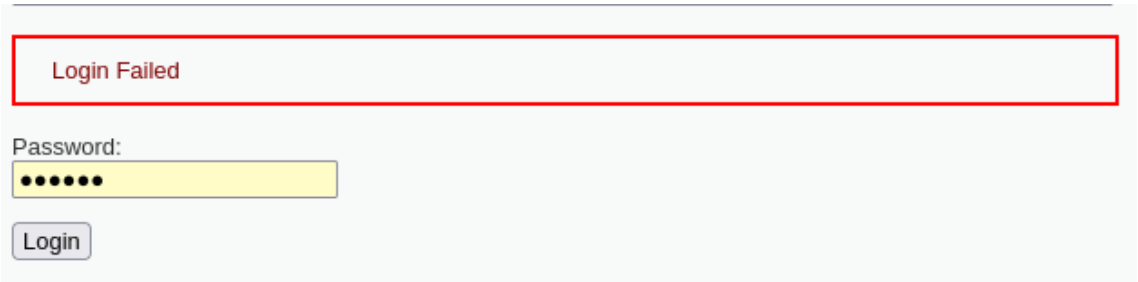
Para verificarlo, el valor interceptado se introdujo en una herramienta de decodificación Base64, utilizando la opción *decode*. Al realizar la decodificación, se obtuvo la contraseña original en texto plano.

Una vez obtenida, se copió y se introdujo en el campo correspondiente de la aplicación, lo que permitió completar correctamente la prueba.



Criptología 5.6.3 Contraseña decodificada

En caso de que la contraseña introducida fuese incorrecta, la aplicación mostraba un mensaje indicando que la autenticación había fallado.



Criptología 5.6.3 Login en caso de que no

Este proceso confirma que la información sensible estaba únicamente codificada y no cifrada, lo que permite recuperar fácilmente los datos originales utilizando herramientas de decodificación.

5.6.4 Recomendaciones

Para mejorar la seguridad criptográfica del sistema se recomienda:

- Utilizar algoritmos de hashing seguros como bcrypt, Argon2 o PBKDF2.

- Implementar salt aleatorio para cada contraseña almacenada.
- Evitar el uso de algoritmos obsoletos como MD5 o SHA1 para el almacenamiento de contraseñas.
- Establecer políticas de contraseñas robustas para los usuarios.
- Limitar los intentos de autenticación para evitar ataques automatizados.

5.6.5 Relacionadas

OWASP Top 10 A02: Cryptographic Failures

CWE CWE-327 – Use of a Broken or Risky Cryptographic Algorithm

CAPEC CAPEC-55 – Rainbow Table Password Cracking